

Novel Clients Generalization in Federated Learning with Hypernetworks and Local Memorization

Ahmad Sidani
Politecnico di Torino
Torino, Italy

s312919@studenti.polito.it

Giuseppe Acquaviva
Politecnico di Torino
Torino, Italy

s318987@studenti.polito.it

Marco Mungai Coppolino
Politecnico di Torino
Torino, Italy

s333969@studenti.polito.it

Abstract

Federated learning is a collaborative machine learning method where multiple devices train a shared model locally, without exchanging data, enhancing privacy and security. In this work, we explore the standard federated scenario and address common data statistical and system heterogeneity challenges, assessing their impact on training. By doing multiple experiments on both IID and non-IID data distributions, we evaluate the performance of the FedAvg algorithm, a widely used method for federated learning that relies on iterative model averaging. Additionally, we investigate the personalized FL task, which aims to improve the adaptation of the global model to individual clients' characteristics. Our contribution addresses the challenge of generalizing to unseen clients by proposing pFedHN-kNN, a method that combines two state-of-the-art FL algorithms: pFedHN and kNN-Per. Specifically, kNN-Per enhances the inferences of the personalized models, trained following the pFedHN method, by leveraging similarities between new clients and previously encountered ones, thereby improving model adaptation and generalization in federated settings.

1. Introduction

Federated Learning (FL) has emerged as a promising paradigm for training machine learning models on decentralized data, addressing privacy concerns associated with traditional centralized training approaches. In federated learning, multiple clients, such as mobile devices or organizations, collaboratively train a shared model without directly sharing their local data. This approach not only enhances privacy but also leverages the computational power distributed across clients. However, federated learning encounters significant challenges due to the diverse nature of data and varying system capabilities across different clients.

One primary concern stems from statistical heterogene-

ity, where data distributions vary significantly between clients. This heterogeneity arises from clients often possessing data specific to their individual usage patterns, geographical locations, or preferences. Consequently, training a single global model that generalizes well across all clients becomes challenging. Another challenge arises from system heterogeneity, where clients have varying levels of computational resources, storage capacities, and communication bandwidths. This variation makes it difficult to ensure efficient and fair training, as resource-constrained clients might lag behind, potentially hindering the overall learning process.

FedAvg [4], a foundational algorithm in FL, addresses the challenges of decentralized training by iteratively averaging locally updated models from participating clients. In each round, clients receive a copy of the global model, update it using their local data, and send these updates back to a central server. The server aggregates these updates, typically through weighted averaging, and applies them to the global model. This process repeats until the global model converges. While FedAvg has shown promise in various applications, its performance can be significantly impacted by statistical heterogeneity.

To mitigate the limitations of global models in statistically heterogeneous settings, Personalized FL (PFL) has gained traction. PFL aims to provide each client with a tailored model better suited to its unique data distribution while still benefiting from collaborative learning across all clients. Various PFL methods have been proposed, employing techniques such as meta-learning, multi-task learning, and local fine-tuning. Despite these advancements, a persistent challenge in PFL is ensuring its generalization ability of the model to unseen clients, i.e., clients not encountered during the training phase.

1.1. Contributions

To tackle the generalization problem, this paper introduces a novel approach to PFL, termed pFedHN-kNN, which combines the strengths of two state-of-the-art FL

algorithms: pFedHN [6] and kNN-Per [3]. pFedHN, or personalized Federated HyperNetworks, employs a central hypernetwork, with the purpose of modifying the weights of the global model different for each client based on a unique embedding vector. By sharing the hypernetwork parameters, pFedHN facilitates effective information sharing among clients while maintaining the flexibility of individualized models. On the other hand, kNN-Per, or k-Nearest Neighbors Personalized Federated Learning, augments a global model (often trained using FedAvg) with a local k-nearest neighbors (kNN) model at each client. This local kNN model operates on a shared representation provided by the global model, leveraging similarities between data points to enhance predictions.

In essence, pFedHN-kNN seeks to improve the generalization capabilities of PFL models, particularly for unseen clients, by combining pFedHN’s strength in generating personalized models with kNN-Per’s ability to adapt to new data distributions through local memorization. The integration of these two techniques aims to address the limitations of existing PFL approaches and enhance their performance in realistic scenarios where data heterogeneity and the presence of unseen clients are prevalent.

2. Related Work

2.1. Federated Average

Federated Averaging (FedAvg), introduced by McMahan et al. [4], is a fundamental algorithm in federated learning that employs iterative model averaging for distributed training. It is specifically designed to train deep networks efficiently in decentralized and potentially sensitive data environments. In this framework, data sources are treated as clients, and the algorithm’s objective is to collaboratively learn a shared global model.

2.2. Personalized Federated Learning

Personalized Federated Learning (PFL) aims to improve upon the limitations of standard FL approaches when clients have heterogeneous data by training models tailored to each client while still benefiting from collaborative learning. PFL techniques aim to address the challenge of training a single global model, which may not be optimal when clients have diverse data distributions. In this setting, in fact, each client $m \in [K]$ has its own local dataset $\mathcal{S}_m = \{s_m^{(i)} = (\mathbf{x}_m^{(i)}, y_m^{(i)}), 1 \leq i \leq n_m\}$ with n_m samples drawn i.i.d from a distribution $\mathcal{D}_m$ over the domain $\mathcal{X} \times \mathcal{Y}$

2.2.1 Hypernetworks

Hypernetworks offer a novel approach to Personalized Federated Learning (PFL) [6]. Hypernetworks are deep neural networks that output the weights of another network,

conditioning on its input [6]. Intuitively, hypernetworks (HNs) simultaneously learn a family of target networks. Let $h(\cdot; \varphi)$ denote the hypernetwork parameterized by φ and $f(\cdot; \theta)$ the target network parameterized by θ . The pFedHN framework employs a central hypernetwork model to generate a set of personalized models, one for each client [6]. This architecture enables effective parameter sharing among clients while maintaining the ability to produce distinct personalized models.

The hypernetwork is located at the server and acts on a client descriptor v_m . The descriptor can be a trainable embedding vector for the client or fixed, provided that a good client representation is known a priori. Given v_m , the HN outputs the weights for the m -th client $\theta_m = \theta_m(v_m) := h(v_m; \varphi)$. Hence, the HN h learns a family of personalized models $\{h(v_m; \varphi)\}_{m \in [K]}$. pFedHN provides a natural way for sharing information across clients while maintaining the flexibility of personalized models by sharing the weight parameters.

2.2.2 Local Memorization

Recent research [3] has shown the efficacy of augmenting neural networks with kNN for improved performance. kNN-based methods can effectively capture local distribution shifts by memorizing rare patterns. Combining kNN with a global model, as in kNN-Per, leverages the global representation while capturing client-specific nuances. This process starts by extracting, from one of the layer of the global model φ , a shared representation ϕ_{h_S} of the data $\mathbf{x}$. The representation is used to build a datastore $(\mathcal{K}_m, \mathcal{V}_m)$, that is queried during inference to produce a local hypothesis $h_{\mathcal{S}_m^{(k)}}$ by using the kNN. Such hypothesis is finally interpolated with the original hypothesis of the global model h_S to produce the final prediction [3] :

$$h_{m, \lambda_m}(\mathbf{x}) \triangleq \lambda_m \cdot h_{\mathcal{S}_m^{(k)}}^{(k)}(\mathbf{x}) + (1 - \lambda_m) \cdot h_S(\mathbf{x}). \quad (1)$$

3. pFedHN-kNN Algorithm

The pFedHN-kNN algorithm integrates the principles of personalized Federated HyperNetworks (pFedHN) with k-Nearest Neighbors Personalized Federated Learning (kNN-Per) to enhance model adaptation and generalization across heterogeneous clients in federated learning scenarios. As discussed in [3] and [6], both kNN-Per and pFedHN allow new clients, arriving post-training, to efficiently learn personalized models without requiring additional training. In kNN-Per, new clients access the global model to build a local datastore for the kNN model, which generates final predictions. In pFedHN, the model learns a meta-model over client distributions during training, enabling efficient generalization with minimal tuning steps and reduced computational effort. This is achieved by freezing the weights of the global model, φ , and optimizing only

Table 1. Datasets and models.

DATASET	TASK	CLIENTS	TOTAL SAMPLES	MODEL
CIFAR-100	IMAGE CLASSIFICATION	100	60,000	CIFAR-LENET
SHAKESPEARE	NEXT-CHARACTER PREDICTION	660	4,226,158	CHARLSTM

the embedding vectors of the new clients, v_{new} . While these methods yield good results without additional training, they do not address how new clients can be fully integrated into the global network. Specifically, we aimed to explore whether, in just a few rounds of training, the model φ could adapt to new clients without compromising performance on the existing clients’ data.

In our Algorithm 1, pFedHN-kNN, we first train an initial hypernetwork $h(\cdot; \varphi)$ using the available clients, following the pFedHN approach. The trained weights are then copied into a second hypernetwork, $h_{new}(\cdot; \varphi)$, which has the same structure as $h(\cdot; \varphi)$ with one difference: the embedding layer is expanded to include additional rows for the embeddings of new clients v_{new} . A brief additional training round allows $h_{new}(\cdot; \varphi)$ to adapt to the new clients while accounting for the existing ones, minimizing overall performance loss. Inspired by skewed participation, we introduced a hyperparameter β to create a weighted probability distribution that controls the bias in the sampling process, balancing the focus between new and old clients. Finally, following the kNN-Per algorithm, we build a datastore $\mathcal{S}_m$ for all clients, which is used to produce the final prediction $h_{m, \lambda_m}(\mathbf{x})$.

Algorithm 1 pFedHN-kNN

Train First Hypernetwork $h(\cdot; \varphi)$ using available clients with pFedHN
Load $h(\cdot; \varphi)$ weights on $h_{new}(\cdot; \varphi)$
Create bias sampling distribution using β
while $r < 100$ **OR** Initial Accuracy Reached **do**
 Sample client m following the weighted distribution
 Perform local training on client m
 Update $h_{new}(\cdot; \varphi)$
end while Use $h_{new}(\cdot; \varphi)$ and kNN-Per to produce final inference

4. Experimental Setup

We conducted two baseline centralized experiments on the CIFAR-100 and Shakespeare datasets. Subsequently, we evaluated FedAvg [4] for federated learning on these datasets: language modeling using the Shakespeare dataset [1] and image classification using CIFAR-100. Additionally, we assessed pFedHN-kNN on the CIFAR-

100 image classification task. The global model h_S was trained with pFedHN [6] and subsequently utilized the kNN-Per method introduced by [3]. The code for these experiments is available at https://github.com/MarcoMungaiCoppolino/Federated-Learning-Group_0.

4.1. Datasets

In a Federated Learning environment, each client works with its own distinct set of training samples. As outlined in Table 1, for Shakespeare it is already provided a pre-defined partition [1]. Instead, for CIFAR-100, we created $K = 100$ disjoint subsets for both the training and test datasets. Moreover, to explore various scenarios of statistical heterogeneity, we applied two different partitioning strategies:

- **IID Sharding:** We grouped all the samples by class, then divided each class’s data into $K = 100$ equal portions. These portions, or shards, were combined from different classes to create each client’s dataset. Since CIFAR-100 has an equal number of images per class, this sharding method ensures uniform distribution, with all clients receiving subsets of equal size. This process is applied to both the training and test datasets. For the training dataset, 20% of the data is set aside to create validation subsets for the clients.
- **non-IID Sharding:** To simulate statistical heterogeneity each client receives a subset containing samples from a fixed number of classes, denoted by N_c . Different values of N_c [1,5,10,50] are used to regulate the data heterogeneity. Each class in the dataset is associated with N_c clients. This is done by first randomly assigning each class to a distinct client, ensuring that every class has at least one client. Additional clients are then assigned to each class through random selection, while ensuring that no client is assigned to the same class more than once. As a result, each client is associated with N_c classes, and each class is distributed across N_c clients.

4.2. Models

For CIFAR-100 classification tasks, the CIFAR-LENET architecture from the work by Real-World et al. [2] was employed. This architecture consists of two convolutional lay-

ers, each employing 5×5 kernels with 64 channels, followed by 2×2 max-pooling layers. Subsequently, two fully-connected layers with 384 and 192 channels respectively were employed, culminating in a softmax classifier for final classification.

For Shakespearean text analysis, the foundational model utilized was a stacked LSTM structure, as detailed in the study by Adaptive et al. [5]. The CharLSTM model specifically incorporates an embedding layer that transforms character indices into dense vectors of dimension 8. These embeddings are processed through a stacked LSTM with 2 layers, each comprising 100 hidden units. The LSTM layers sequentially process the input sequences, and the final hidden state from the last time step is extracted. This hidden state undergoes further processing through a fully connected layer that outputs 80 classes, ultimately utilized in a softmax classifier for classification tasks.

5. Experiments

5.1. Centralized baseline

In the **centralized experiment** on CIFAR-100, training was conducted for 150 epochs using the SGDM optimizer with a learning rate of 0.01, $\beta = 0.9$, weight decay set to 1×10^{-3} , and a cosine annealing scheduler. For the Shakespeare dataset, training was conducted for 25 epochs using the SGDM optimizer with a learning rate of 0.01, $\beta = 0.9$, weight decay set to 1×10^{-4} .

Table 2. Centralized baseline

Dataset	Clients	Classes	Centralized Accuracy
CIFAR-100	100	100	57.34%
Shakespeare	660	80	55.65%

For CIFAR-100, the loss and accuracy were calculated as indicated in Figures 1a and 1b, showing that the values converge after 150 epochs. The tuning was performed on the learning rate and weight decay, using a validation set created from the training dataset with a 80, 20 split. Other learning schedulers were tried without significant results. For Shakespeare, the loss and accuracy were calculated as indicated in Figures 1c and 1d, showing that the values converge after 25 epochs. The tuning was performed on the learning rate and weight decay, instead a cosine annealing scheduler was tried but did not yield any improvement. Results showed in Table 2

5.2. FL baseline

In the **federated learning (FL) baseline experiment**, we implemented the algorithm as outlined by [4]. Specifically, we set the number of clients to $K=100$ and partitioned the CIFAR-100 dataset between them. We evaluated the FedAvg algorithm over the course of 2000 rounds, without

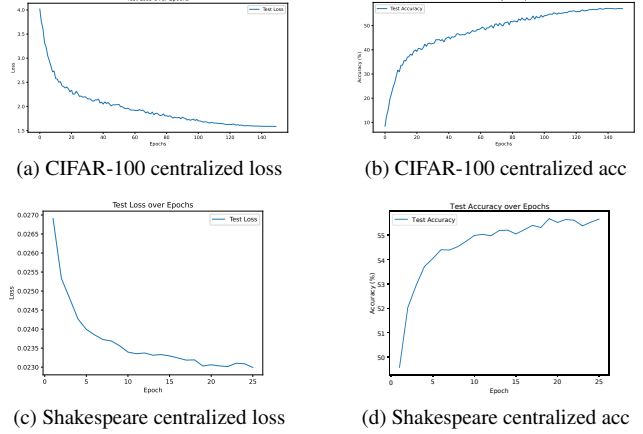


Figure 1. test loss and test accuracy CIFAR-100 and Shakespeare.

applying any learning rate scheduling. At each communication round, a fraction ($C = 0.1$) of the clients was selected. Each client performed J local training steps per round.

5.2.1 Impact of Client Participation

The training dataset was initially partitioned in an independent and identically distributed (IID) manner. In the first experiment, we investigated the effect of varying the hyperparameter γ of the Dirichlet distribution, which controls the level of skewness in the data distribution. Specifically, lower values of γ lead to increased heterogeneity in client sampling, reflecting more pronounced data imbalance.

Table 3. Test Accuracy for Uniform vs Skewed Participation, CIFAR100 IID

	Uniform	Skewed		
		Gamma 0.1	Gamma 0.5	Gamma 0.7
Values	24.34%	22.09%	23.84%	23.08%

As shown in Table 3, there was a slight reduction in test accuracy when the data distribution becomes more skewed. For $\gamma = 0.1$, which represents the highest level of data imbalance, accuracy decreases from 24.34% (uniform) to 22.09%. However, for $\gamma = 0.5$ and $\gamma = 0.7$, the accuracy is closer to the uniform sampling scenario, suggesting that moderate skewness does not drastically affect performance.

Table 4. Test Accuracy for Uniform vs Skewed Participation, Shakespeare IID

	Uniform	Skewed		
		Gamma 0.1	Gamma 0.5	Gamma 0.7
Accuracy	31.19%	31.41%	39.11%	42.70%

In contrast to CIFAR100, Table 4 demonstrates a notable improvement in test accuracy for the Shakespeare IID dataset under skewed data participation. With $\gamma = 0.1$, accuracy improves slightly to 31.41%, but for $\gamma = 0.5$ and $\gamma = 0.7$, accuracy increases significantly, reaching 39.11% and 42.70% respectively. This suggests that the skewed data distribution in this context might have been more aligned with the nature of the task, possibly leading to better specialization of client models.

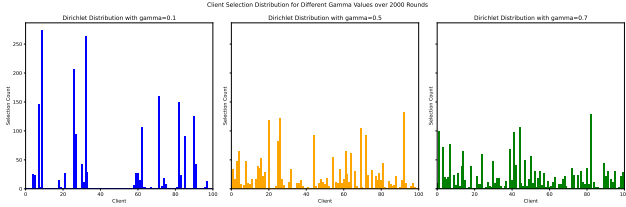


Figure 2. Dirichlet Distribution

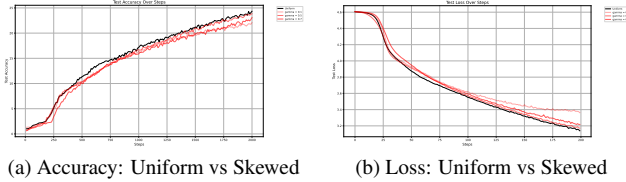


Figure 3. Comparison of test accuracy and loss for uniform vs skewed participation on CIFAR100 IID.

The different sampling distributions are shown in Figure 2, while the performance comparisons between uniform and skewed sampling strategies are illustrated in Figures 3a and 3b. As seen in the accuracy plot (Figure 3a), the CIFAR100 dataset shows a slight performance degradation when the participation becomes skewed. However, the loss plot (Figure 3b) indicates that the model with skewed participation tends to converge similarly to the uniform participation scenario, despite minor differences in test accuracy. This suggests that while skewed participation introduces data imbalance, it does not drastically hinder model convergence.

5.2.2 Heterogeneous distributions

In this subsection, we analyzed the performance of Federated Averaging (FedAvg) under non-IID data distributions for both CIFAR-100 and Shakespeare datasets. Figure 4 illustrates the performance of FedAvg on CIFAR-100 with varying levels of non-IID partitioning, compared to the IID baseline. The non-IID settings were created by varying the number of classes per client (N_c), reflecting different degrees of data heterogeneity. Figure 5a shows that increasing (N_c) leads to a more balanced distribution of data across

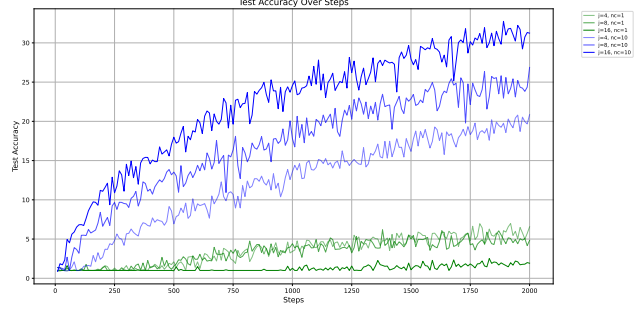


Figure 4. Performance in non-IID settings for CIFAR-100.

clients, which reduces heterogeneity and improves performance.

Table 5. CIFAR-100, Non-IID, Uniform Participation, Test Accuracy

	NC=1	NC=5	NC=10	NC=50
j=4	6.56%	15.85%	20.85%	23.69%
j=8	4.83%	20.95%	26.87%	31.4%
j=16	1.92%	26.07%	31.24%	39.62%

The results from Table 5 confirm that performance degrades as the data distribution becomes more unique in the non-IID setting, with the performance improving as (N_c) increases. Clients with fewer classes exhibit lower test accuracy, as they struggle to generalize better compared to those with more classes. The results from the skewed participation runs in non-IID setting are shown in Table 7. Across all the N_c runs, the weighted sampling distribution hasn't produced much difference in performance between the diverse values of gamma.

The evaluation of the Shakespeare dataset, known for its non-IID nature due to client-specific character dialogues, is presented in Table 6.

Table 6. Uniform vs Skewed Participation Test Accuracy, Shakespeare Non-IID

	Uniform	Skewed		
		Gamma 0.1	Gamma 0.5	Gamma 0.7
J=4	30.97%	31.31%	38.8%	42.56%
J=8	38.36%	43.79%	46.66%	48.67%
J=16	42.80%	46.73%	49.93%	50.14%

This table demonstrate that skewed participation, particularly with lower gamma values, enhances performance relative to uniform participation. Additionally, increasing the number of local steps (J) shows improved accuracy in both IID and non-IID scenarios as shown in Tables 4, and 6. Figure 5b highlights the data distribution resulting from the non-IID sharding of the Shakespeare dataset,

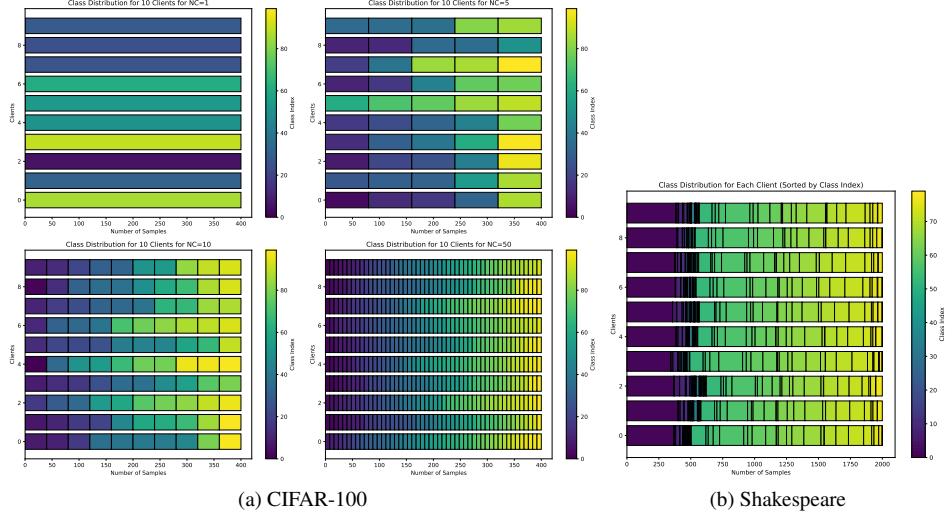


Figure 5. Comparison between the data distributions resulting from CIFAR-100 data sharding procedure and from Shakespeare splits.

Table 7. CIFAR-100, Non-IID, Skewed Participation, Test Accuracy

GAMMA	NC=1			NC=5			NC=10			NC=50		
	0.1	0.5	0.7	0.1	0.5	0.7	0.1	0.5	0.7	0.1	0.5	0.7
J=4	7.09%	4.96%	5.96%	17.04%	17.24%	16.56%	20.01%	21.76%	18.35%	24.28%	24.70%	24.04%
J=8	3.8%	5.55%	4.66%	22.02%	21.87%	20.4%	24.98%	25.62%	26.27%	33.85%	33.89%	33.02%
J=16	2.80%	3.15%	2.73%	23.07%	23.75%	25.13%	30.33%	28.32%	30.61%	40.05%	38.37%	39.29%

emphasizing the diverse distribution of text classes among clients. This further illustrates the impact of data distribution on performance. In summary, our findings indicate that data heterogeneity notably affects the convergence of FedAvg, with more homogeneous distributions (larger (N_c) or less skewed participation) resulting in better performance. Nonetheless, increasing J partially mitigates the performance gap between IID and non-IID scenarios, as demonstrated in both CIFAR-100 and Shakespeare datasets.

5.3. Personalized FL

In the experiments regarding the pFedHN-kNN, we used the same global model as the FL baseline. For the hypernetwork, we followed the architecture described in [6], which is a fully-connected neural network featuring three hidden layers and multiple linear heads for each target weight tensor. Due to computational and time constraints, we chose to concentrate on non-IID setting, testing exclusively on CIFAR-100 data, and using the same sharding approach as the FL baseline, $N_c = [1, 5, 10]$. Specifically, we divided the 100 clients into 90 old clients for the initial training phase and introduced the remaining 10 new clients in the second round of training. Additionally, for simplicity, dur-

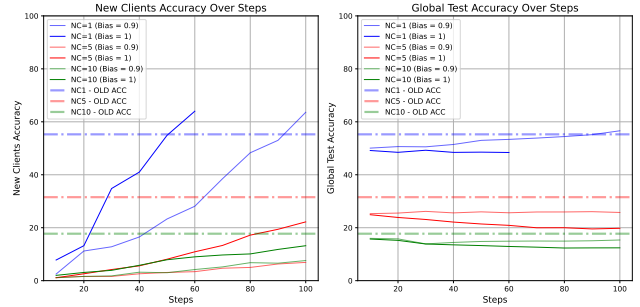


Figure 6. Test Accuracy between the Generalization of new clients and the Global test accuracy

ing the initial training phase, we limited our experiments to models trained with uniform client participation. Following the pFedHN algorithm described in [6], at each step, only a single client is selected for local training, unlike the FedAvg method where a fraction $C = 0.1$ of clients is chosen. To ensure comparable computation time with the FL baseline, rather than selecting $C \times K$ clients and having them perform J rounds of local training, we selected one client and allowed him to perform $J \times C \times K$ rounds. The

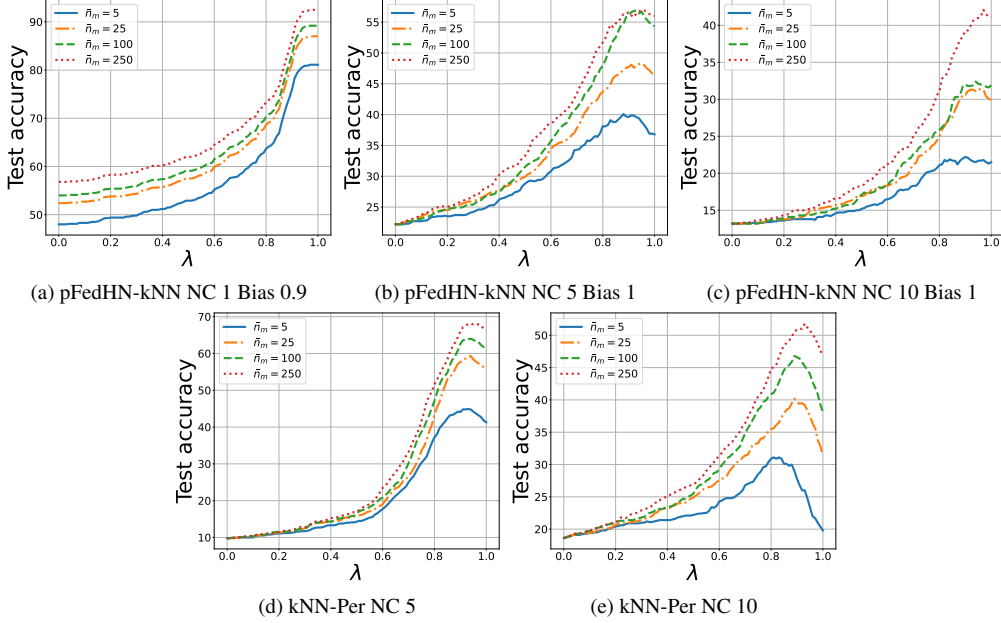


Figure 7. Comparison between the data distributions resulting from CIFAR-100 data sharding procedure and from Shakespeare splits.

initial plan was to train the first hypernetwork $h(\cdot; \varphi)$ for 1900 rounds, reserving the final 100 rounds for the second phase of training. However, during experiments, we observed that for $N_c = 1$, the hypernetwork reached high accuracy much earlier than expected. As a result, we decided for the case to stop the initial training after 500 rounds and proceed with the generalization phase, leaving the comparisons for higher N_c . In each generalization run, the training of $h_{new}(\cdot; \varphi)$ was conducted for up to 100 rounds, or terminated early if the accuracy on the new clients exceeded the accuracy achieved at the end of the initial training. The biases used in the study are $\beta = 1$, where only new clients were selected for training, and $\beta = 0.9$, which ensured that in each step there was an equal probability of selecting either an old or a new client. The results of the generalization training are shown in Figure 6. As expected, the runs with a bias of 1 achieved higher accuracy on the new clients more quickly, while the runs with a bias of 0.9 were better suited for a more balanced integration, resulting in a global model that remained closer to the original. Of all the runs, the one with $N_c = 1$ showed the more significant improvement, being able to even surpass the original accuracy, in the given amount of time

After the generalization phase, we used $h_{new}(\cdot; \varphi)$ to compute the fixed-length representation ϕ_{h_S} , required for creating the datastore used during inference. For each client dataset $\mathcal{S}_m$ the hypernetwork generates the model weights specific to that client’s data. The output of the second fully connected layer of the personalized model is then used as the shared representation. Initially, we attempted to use the

output of the last convolutional layer, as suggested in [3], but encountered dimension incompatibility issues. After building the datastore, in similar fashion to what done in the paper, we studied the performance of the kNN inference across different local dataset client sizes n_m and using different λ_m . In the Figure 7 are shown the results of the kNN inference done on some of the generalized runs (7a, 7b, 7c). The runs showed a significant improvement in accuracy regardless of N_c . A common trend, particularly with larger local datasets and smaller N_c was the strong bias of the clients towards the inference $h_{\mathcal{S}_m}^{(k)}(\mathbf{x})$, resulted from the kNN model. To have a fair comparison with the original kNN-Per algorithm, we reused the training done with the FedAvg method in the FL baseline, and then tested the performance on the kNN augmentations. As Shown in 7d and 7e although the FedAvg algorithm achieved lower accuracy compared to pFedHN training, the kNN approach was able to enhance its performance, surpassing the accuracy achieved by our algorithm (7b, 7c). This means that the integration of the local memorization method requires further studies in the context of already personalized approaches like pFedHN.

6. Conclusion

The integration of personalized Federated HyperNetworks (pFedHN) with k-nearest neighbors personalized federated learning (kNN-Per) through the pFedHN-kNN algorithm has shown significant improvements in addressing data heterogeneity and adapting to unseen clients. However, several promising directions remain for future exploration

based on this study’s findings. First, one area that warrants further investigation is the initialization of embedding vectors introduced in the new HyperNetwork layer. In preliminary experiments we attempted to initialize these embeddings by averaging the existing ones, but this approach led to poorer generalization and was therefore excluded from the final report. Additionally, as the task of integrating new clients continues to evolve, the HyperNetwork architecture may need to scale. Simply updating the embedding layer might not suffice if the number of new clients continues to grow, suggesting that more extensive modifications to the model may be necessary to maintain performance over time. Finally, as discussed at the end of the Personalized paragraph, even if the kNN improved the accuracy after the generalization, the better results of the kNN-Per prove that the inference process requires a deeper analysis.

References

- [1] Sebastian Caldas, Peter Wu, Tian Li, Jakub Konečný, H. Brendan McMahan, Virginia Smith, and Ameet Talwalkar. LEAF: A benchmark for federated settings. *CoRR*, abs/1812.01097, 2018. [3](#)
- [2] Tzu-Ming Harry Hsu, Hang Qi, and Matthew Brown. Federated visual classification with real-world data distribution. *CoRR*, abs/2003.08082, 2020. [3](#)
- [3] Othmane Marfoq, Giovanni Neglia, Laetitia Kameni, and Richard Vidal. Personalized federated learning through local memorization. *CoRR*, abs/2111.09360, 2021. [2](#), [3](#), [7](#)
- [4] H. Brendan McMahan, Eider Moore, Daniel Ramage, and Blaise Agüera y Arcas. Federated learning of deep networks using model averaging. *CoRR*, abs/1602.05629, 2016. [1](#), [2](#), [3](#), [4](#)
- [5] Sashank J. Reddi, Zachary Charles, Manzil Zaheer, Zachary Garrett, Keith Rush, Jakub Konečný, Sanjiv Kumar, and H. Brendan McMahan. Adaptive federated optimization. *CoRR*, abs/2003.00295, 2020. [4](#)
- [6] Aviv Shamsian, Aviv Navon, Ethan Fetaya, and Gal Chechik. Personalized federated learning using hypernetworks. *CoRR*, abs/2103.04628, 2021. [2](#), [3](#), [6](#)